CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 1: Array Operations: Traversal, Rotation, and Frequency Analysis

Practical Aim:

To implement fundamental array operations.

Problem 1 :

A bakery prepares n items every morning and places them in a display row. At the end of each hour, the leftmost item is moved to the rightmost position to make room for fresh stock at the front. Given the initial row and the number of hours h, print the final display order. Describe the approach you used. How does it behave when h is very large, say 10 million? Is there a way to get the correct result without performing the operation h times?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-1

Output:



Problem Faced:

One of the main challenges in this problem is handling a large number of rotations efficiently. If the leftmost item is moved to the end for every hour, the program may need to perform millions of rotations when the value of **h** is very large. This increases the execution time and makes the solution inefficient.

Outcome/Conclusion:

To overcome this challenge, we use the observation that after n rotations (where n is the number of items), the display returns to its original order. Therefore, instead of performing h rotations, we calculate h % n and perform only the effective rotations. This reduces unnecessary operations and allows the final display order to be determined efficiently, even when h is very large.

Problem 2 :

A library issues books to students and records each book's ID every time it is borrowed. At the end of the month, the librarian wants to find all books that were borrowed more than once, as those need priority restocking. Given the borrowing log, print all such book IDs. Describe the approach you used. How many times does your solution scan the data? If the library had 100,000 borrow records, would your approach still be practical? Can you think of a way that requires fewer passes?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-1

Output:



Problem Faced:

The main challenge is efficiently identifying book IDs that appear more than once in a large borrowing log. Using a nested loop to compare each book ID with every other ID would require a large number of comparisons, making the solution slow and inefficient for datasets containing thousands of records.

Outcome/Conclusion:

To solve this problem efficiently, a frequency-counting approach can be used. Each book ID is scanned and its occurrence count is stored in a hash map or similar data structure. After counting the frequencies, the IDs with a count greater than one are printed. This approach requires only a small number of passes through the data and works efficiently even for very large borrowing logs.

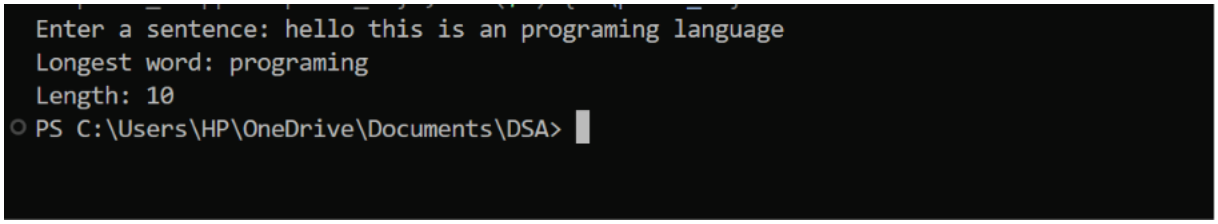
Problem 3 :

A school newspaper editor wants to highlight the most impressive word from a submitted article on the front page. The word is chosen simply by length — the longest one wins. Given a sentence, print the winning word and how many letters it has. Describe the approach you used. What does your solution do when two words have the same length? Is that behavior intentional, and can you think of a way to handle it explicitly?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-1

Output:



Problem Faced:

The main challenge is correctly identifying the longest word in the given sentence. Since a sentence may contain multiple words of different lengths, the program must compare the length of each word and keep track of the longest one found. Another challenge arises when two or more words have the same maximum length.

Outcome/Conclusion:

To solve this problem, the sentence is split into individual words, and each word is checked one by one. The length of each word is compared with the current longest word. If a longer word is found, it becomes the new longest word. In the case of two words having the same length, the current implementation keeps the first word encountered. This behavior is simple and consistent, though it can be modified to explicitly handle ties by displaying all longest words or selecting the last occurrence.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation:

[Problem 1] The display order repeats itself after a certain number of hours regardless of how large h gets. Without trying any specific example, explain why this repetition is inevitable and what determines when it first repeats.

Sol: The display order repeats after n rotations because all items return to their original positions. Therefore, the first repetition occurs after n rotations.

[Problem 2] If the same book ID appears three times in the log, should it appear once or three times in your output? Does your solution handle this correctly, and how would you verify it?

Sol: If a book ID appears three times, it should be printed only once. The frequency map ensures this by checking counts instead of printing every occurrence.

[Problem 3] The editor later decides that punctuation attached to a word (e.g. “hello,” or “great!”) should not count toward its length. How would this change your solution, and at which step would you handle it?

Sol: If punctuation should not be counted, remove punctuation from each word before calculating its length. This ensures accurate results.